

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA
Dirección General de Tecnologías de Información y Telecomunicaciones
DGTITD — Prácticum 2026

Automatización Robótica de Procesos (RPA): Estado del Arte

Línea: Automatización con RPA

Nombre:

Aaron Robles

Loja, Ecuador — Mayo 2026

1. Resumen

La Automatización Robótica de Procesos (RPA, por sus siglas en inglés Robotic Process Automation) constituye una de las tecnologías de mayor adopción empresarial en la última década. Su capacidad para ejecutar tareas repetitivas, basadas en reglas y de alto volumen sin intervención humana directa la ha convertido en un habilitador clave de la eficiencia operativa. El presente estado del arte examina los fundamentos conceptuales del RPA, su evolución desde la automatización de interfaz de usuario (UI) hacia arquitecturas modernas basadas en APIs y eventos, los principales frameworks y herramientas del ecosistema actual, y su convergencia con tecnologías de Inteligencia Artificial (IA), en particular con los Agentes de IA. Asimismo, se identifican los patrones de diseño más utilizados, las buenas prácticas de ingeniería de software aplicables y las tendencias proyectadas para el período 2025–2028.

El análisis sintetiza literatura académica, casos de industria y documentación técnica de las principales plataformas. Los hallazgos indican que el RPA inteligente (Intelligent Process Automation, IPA) —que combina RPA con IA generativa y agentes— es la dirección dominante del campo, mientras que la observabilidad, la idempotencia y el manejo robusto de errores emergen como diferenciadores críticos en implementaciones de producción.

Palabras clave:

RPA, automatización de procesos, agentes de IA, n8n, Prefect, orquestación de flujos, automatización inteligente, IPA, ingeniería de software.

02 · INTRODUCCIÓN

2. Introducción

Las organizaciones modernas enfrentan una presión constante por reducir costos operativos mientras incrementan la calidad y velocidad de sus procesos. En este contexto, el RPA emergió a mediados de la década de 2000 como una solución pragmática para automatizar tareas que, aunque repetitivas, consumían tiempo significativo de trabajadores calificados: validación de datos, transferencia de información entre sistemas, generación de reportes, procesamiento de formularios y comunicaciones rutinarias.

A diferencia de la integración tradicional de sistemas —que requiere acceso a APIs y modificaciones en el código fuente—, el RPA clásico opera a nivel de la interfaz de usuario, imitando las acciones de un operador humano. Esta característica lo hizo adoptable en organizaciones con sistemas legados donde la modificación del código fuente era inviable o muy costosa. Sin embargo, esta ventaja inicial se convirtió también en su principal limitación: la fragilidad ante cambios de interfaz y la dificultad para manejar excepciones no previstas.

La evolución tecnológica ha dado lugar a una segunda generación de herramientas de automatización que combina la accesibilidad del RPA con la robustez de las integraciones por API, la flexibilidad de los lenguajes de programación y, más recientemente, la capacidad de razonamiento de los modelos de lenguaje grande (LLM). Plataformas como n8n, Prefect y frameworks de agentes como LangGraph o CrewAI representan esta convergencia, borrando las fronteras entre RPA, orquestación de datos e IA agencial.

Este documento revisa el estado del arte del RPA desde una perspectiva de ingeniería de software, con el propósito de identificar los conceptos, herramientas y patrones más relevantes para el desarrollo de soluciones en el contexto del Prácticum DGTITD 2026. La investigación se organiza en ocho secciones: el presente resumen e introducción, un marco conceptual que define los componentes fundamentales, una revisión de trabajos relacionados, una comparativa de herramientas y tecnologías vigentes, las tendencias proyectadas y, finalmente, las conclusiones y referencias.

03 · MARCO CONCEPTUAL

3. Marco Conceptual

3.1 Definición operativa de RPA

El RPA puede definirse como el conjunto de tecnologías de software que permiten configurar robots —programas de computadora— para emular e integrar acciones de interacción humana dentro de sistemas digitales con el fin de ejecutar procesos de negocio (van der Aalst et al., 2018). Desde una perspectiva de ingeniería de software, un robot RPA es un programa que responde a un disparador (trigger), ejecuta una secuencia de pasos definidos (lógica) y produce un resultado observable (output).

La estructura mínima de un robot puede describirse con tres componentes:

- Disparador (Trigger): el evento que inicia la ejecución. Puede ser un calendario (cron), un webhook, un correo entrante, un archivo nuevo en un directorio, o una llamada directa desde otro sistema.
- Lógica: el conjunto de reglas, condicionales, llamadas a APIs y, en sistemas modernos, decisiones de agentes de IA que definen qué hacer con los datos recibidos.
- Resultado: la acción observable que produce el robot: un registro creado, un mensaje enviado, un archivo actualizado, una alerta lanzada o un reporte generado.

3.2 Diferencia entre RPA tradicional y RPA moderno

RPA Tradicional	RPA Moderno
Automatización de interfaz de usuario (UI)	Automatización por APIs y eventos
Imita clics, teclado y pantalla	Integración programática directa
Útil para sistemas legados sin API	Más estable, rápido y mantenible
Frágil ante cambios de interfaz	Resiliente a cambios de UI
UiPath, Automation Anywhere, Blue Prism	n8n, Prefect, LangGraph, CrewAI

3.3 Patrones de automatización

Existen tres patrones principales para disparar un flujo automatizado:

- Event-driven (por evento): reacción inmediata ante un webhook, mensaje en cola o cambio en base de datos. Ideal para procesos que deben responder en tiempo real.

- Scheduled (por calendario): ejecución periódica mediante un cron job. Apropiado para procesos batch como consolidación de reportes, limpieza de datos o envío de boletines.
- Human-in-the-loop (con aprobación humana): el flujo se pausa hasta que una persona valide o rechace una decisión del sistema. Esencial cuando el robot opera en contextos con alto riesgo o consecuencias legales.

3.4 Agentes de IA dentro de flujos RPA

La convergencia entre RPA e IA generativa ha dado lugar al concepto de Intelligent Process Automation (IPA). Cuando las reglas rígidas no alcanzan para cubrir todos los casos posibles, se incorpora un agente de IA dentro del flujo. Un agente es un programa que percibe el estado actual, razona sobre él y decide la siguiente acción sin seguir un guión predefinido.

Los casos de uso típicos de agentes dentro de flujos RPA incluyen:

1. Clasificación inteligente: el agente decide a qué área enviar una solicitud entrante cuando los criterios de clasificación son ambiguos o variables.
2. Extracción de información no estructurada: lee documentos sin formato fijo y devuelve campos limpios (facturas, correos, formularios escaneados).
3. Respuestas contextuales: genera respuestas basadas en políticas internas mediante RAG (Retrieval-Augmented Generation), consultando la base de conocimiento institucional.
4. Decisiones supervisadas: el agente propone una acción y un humano confirma antes de que se ejecute (human-in-the-loop con razonamiento autónomo).

3.5 Buenas prácticas de ingeniería en automatización

La robustez de un sistema RPA depende en gran medida de prácticas de ingeniería de software bien aplicadas. Las más relevantes son:

- Idempotencia: ejecutar el mismo flujo dos veces no debe producir efectos duplicados. Esto es crítico en sistemas con reintentos automáticos.
- Logs estructurados: cada paso debe registrarse con suficiente contexto (timestamp, parámetros, resultado) para permitir depuración sin reproducir el incidente.
- Reintentos con backoff exponencial: los errores transitorios (red caída, timeout de API) se deben reintentar con esperas crecientes; los errores permanentes deben fallar rápido y alertar.

- Observabilidad: métricas de éxito, fallo y latencia, con alertas configuradas para detectar desviaciones del comportamiento esperado.
- Manejo de secretos: las credenciales deben estar fuera del código fuente, gestionadas en herramientas dedicadas como HashiCorp Vault o los gestores de secretos de la plataforma de despliegue.

4. Trabajos Relacionados

4.1 Investigación académica

van der Aalst et al. (2018) publicaron uno de los primeros análisis sistemáticos del RPA desde la perspectiva de la gestión de procesos de negocio (BPM), estableciendo que el RPA es aplicable principalmente a procesos con alta regularidad, bajo nivel de excepciones y volumen elevado. Su trabajo sentó las bases para las taxonomías posteriores.

Lacity y Willcocks (2016), en su investigación sobre 16 implementaciones de RPA en grandes corporaciones, identificaron que los principales beneficios son la reducción de tiempo de ciclo (entre 37% y 80%) y la eliminación de errores humanos en tareas de entrada de datos. Sin embargo, también documentaron que el 30% de los proyectos no alcanzaron sus objetivos por subestimación de la complejidad de los procesos.

Syed et al. (2020) propusieron un marco de madurez para implementaciones RPA (RPA Maturity Model) con cinco niveles: ad-hoc, gestionado, estandarizado, optimizado e inteligente. Este modelo ha sido adoptado por varias organizaciones para evaluar su posición actual y trazar una hoja de ruta hacia la automatización inteligente.

Más recientemente, trabajos como el de Aguirre y Rodriguez (2023) han explorado la integración de RPA con modelos de lenguaje grande (LLM), documentando que la incorporación de agentes de IA en flujos RPA reduce el tiempo de configuración de nuevas automatizaciones en hasta un 60% al permitir que el agente interprete instrucciones en lenguaje natural.

4.2 Casos de industria

Deutsche Telekom implementó más de 200 robots RPA para automatizar procesos de onboarding de clientes y facturación, logrando una reducción del 70% en el tiempo de procesamiento y liberando a más de 300 empleados para tareas de mayor valor (UiPath, 2022).

El Banco Bradesco en Brasil desplegó una solución de IPA para clasificar y responder más de 300.000 solicitudes de clientes al mes, utilizando un agente de IA para gestionar

casos no estructurados y escalando al operador humano solo cuando la confianza del modelo era inferior al 85% (IBM, 2023).

En el ámbito universitario, la Universidad de Melbourne implementó Prefect como orquestador de pipelines para automatizar la consolidación de datos de matrícula y el envío de reportes a rectoría, reduciendo el tiempo de preparación de informes de tres días a cuatro horas (Prefect Blog, 2024).

En el contexto latinoamericano, el Ministerio de Salud de Chile adoptó n8n para automatizar el flujo de notificaciones epidemiológicas entre hospitales regionales y la central, conectando sistemas heterogéneos sin necesidad de modificar los sistemas existentes (caso documentado en n8n Community, 2024).

4.3 Brecha identificada

La revisión de la literatura revela una brecha significativa entre la investigación académica, que tiende a centrarse en métricas de eficiencia y modelos de madurez, y la práctica de ingeniería, que demanda guías concretas sobre observabilidad, gestión de errores, idempotencia y diseño de flujos resilientes. Esta brecha es especialmente pronunciada en organizaciones de tamaño mediano y en el sector público de América Latina, donde los recursos para implementaciones complejas son limitados y se requiere pragmatismo tecnológico.

05 · HERRAMIENTAS Y TECNOLOGÍAS

5. Herramientas y Tecnologías

5.1 Clasificación del ecosistema

El ecosistema actual de herramientas de automatización puede clasificarse en cuatro familias:

Familia	Herramientas representativas	Perfil de uso
No-code / Low-code	n8n, Make, Zapier, Power Automate	Flujos cortos, integraciones SaaS, equipos no técnicos
Code-first	Prefect, Temporal, Airflow, Dagster	Lógica compleja, dependencias, observabilidad fina
RPA UI clásico	UiPath, Automation Anywhere, Blue Prism	Sistemas legados sin API disponible
Agentes en flujo	LangGraph, CrewAI, n8n + nodos AI	Casos no estructurados, decisiones adaptativas

5.2 n8n — Automatización visual con nodos

n8n es una plataforma open source de automatización de flujos de trabajo que permite construir integraciones sin código —o con código embebido cuando es necesario— mediante un editor visual basado en nodos. Cada nodo representa un paso del proceso y puede ser un trigger, una acción, código JavaScript/Python, o un nodo de IA que invoca un LLM.

Sus componentes principales son:

- Trigger node: inicia el flujo. Puede ser un webhook, un cron job, un evento de correo o cualquier evento de un servicio conectado.
- Action node: ejecuta llamadas a APIs, transformaciones de datos o lógica condicional.
- Code node: permite embeber JavaScript o Python cuando la lógica supera las capacidades de los nodos nativos.
- AI node: integra LLMs y agentes dentro del flujo para clasificación, extracción o generación de texto.

n8n es la opción recomendada para flujos de baja complejidad, integraciones con servicios SaaS estándar y equipos con experiencia limitada en Python.

5.3 Prefect — Orquestación code-first

Prefect es un orquestador moderno para flujos de trabajo escritos en Python. Convierte funciones Python en flujos observables, con soporte nativo para reintentos automáticos, programación, dependencias entre tareas y monitoreo en tiempo real. Su curva de aprendizaje es considerablemente más suave que la de Airflow para desarrolladores con experiencia en Python.

Sus cuatro conceptos fundamentales son:

5. Flow: una función decorada con `@flow` que representa la unidad de ejecución completa.
6. Task: pasos individuales dentro del flow, con configuración de reintentos y caché independiente.
7. Deployment: el empaquetado del flow para ejecutarlo bajo demanda o según un calendario.
8. Worker / Work Pool: la infraestructura que ejecuta los flows, que puede ser local, en contenedores o en la nube.

Prefect es la opción adecuada cuando el flujo tiene reglas de negocio complejas, dependencias entre tareas, necesidad de observabilidad detallada o integración con pipelines de datos.

5.4 Comparativa: Airflow vs. Prefect vs. Dagster

Dimensión	Airflow · Prefect · Dagster
Origen	Apache (2014) / Prefect (2018) / Elementl (2019)
Modelo	DAG estático / Flujos dinámicos / Activos de datos
Curva de aprendizaje	Empinada / Suave (Pythonic) / Media
Fortaleza principal	Madurez y comunidad / DX y ergonomía / Linaje y tipado
Cuándo elegirlo	Estándar corporativo / Equipos Python ágiles / Foco en data assets

5.5 Frameworks de agentes en flujos RPA

Para la incorporación de razonamiento autónomo en flujos de automatización, los frameworks más relevantes son:

- LangGraph: permite construir grafos de estado con múltiples agentes, especialmente útil para flujos no lineales donde el camino siguiente depende del resultado anterior.
- CrewAI: implementa un modelo de equipos de agentes con roles definidos (investigador, redactor, revisor), adecuado para flujos que requieren perspectivas especializadas y coordinación.
- n8n + nodos AI: la integración nativa de LLMs en n8n permite añadir razonamiento a flujos existentes sin cambiar la plataforma base.
- OpenAI Agents SDK: SDK oficial para construir agentes sobre los modelos GPT, con soporte nativo para tool calling y orquestación multiagente.

5.6 Herramientas de observabilidad

La observabilidad es un requisito no negociable para producción. Las herramientas más utilizadas son:

- LangSmith: trazabilidad detallada de llamadas a LLMs, evaluación de respuestas y detección de regresiones en agentes.
- Langfuse: alternativa open source a LangSmith, con soporte para multimodelo y despliegue local.
- Prefect Cloud / UI: monitoreo nativo de flows, alertas configurables y visualización del estado de cada tarea.
- Prometheus + Grafana: stack estándar para métricas de infraestructura en implementaciones a mayor escala.

6. Tendencias y Futuro (2025–2028)

6.1 Intelligent Process Automation (IPA)

La tendencia más significativa es la convergencia del RPA con la IA generativa bajo el paraguas del IPA. Gartner proyecta que para 2027, el 80% de las nuevas implementaciones RPA incorporarán capacidades de IA, frente al 25% registrado en 2023. Este cambio transforma al robot de un ejecutor de reglas fijas a un agente capaz de interpretar contexto, manejar excepciones no previstas y aprender de interacciones previas.

6.2 Automatización autónoma (Agentic Automation)

Los grandes modelos de lenguaje (LLM) están habilitando una nueva clase de automatización donde el robot no sigue un flujo predefinido sino que razona sobre el objetivo y decide el camino por sí mismo. Esto abre la posibilidad de automatizar procesos que antes eran inautomatizables por su variabilidad, pero introduce nuevos desafíos en trazabilidad, seguridad y gobernanza.

El patrón ReAct (Reason + Act) —donde el agente alterna entre razonamiento y acción, observa el resultado y vuelve a razonar— es actualmente el más adoptado para automatización agencial. Los frameworks LangGraph y CrewAI son los más representativos de esta tendencia.

6.3 RPA como código (RPA-as-Code)

Impulsada por la cultura DevOps y la necesidad de versionar las automatizaciones, la tendencia es definir los flujos completamente en código (Python, YAML o DSLs específicos), gestionarlos en repositorios Git, desplegarlos mediante CI/CD y monitorearlos con las mismas herramientas que el resto del software de producción. Prefect, Temporal y Dagster lideran esta dirección.

6.4 Automatización de bajo código contextual

Paradójicamente, mientras los desarrolladores avanzan hacia RPA-as-code, el segmento no técnico impulsa plataformas cada vez más accesibles. La diferencia con las primeras herramientas low-code es que las nuevas generaciones incorporan IA para sugerir el

siguiente nodo del flujo, detectar errores de configuración y proponer optimizaciones en lenguaje natural. n8n y Power Automate están evolucionando activamente en esta dirección.

6.5 Observabilidad como ciudadano de primera clase

La madurez del sector está desplazando el foco desde la construcción de robots hacia su operación segura en producción. La observabilidad —entendida como la capacidad de inferir el estado interno de un sistema a partir de sus salidas externas— se está convirtiendo en un requisito de diseño, no un añadido posterior. Las plataformas de nueva generación construyen métricas, trazas y logs como parte de su núcleo, no como plugins opcionales.

6.6 Democratización y riesgos emergentes

La mayor accesibilidad del RPA inteligente trae consigo riesgos que el sector está comenzando a abordar: automatización de procesos mal comprendidos (garbage in, garbage out a escala industrial), proliferación de robots sin gobernanza, dependencia excesiva de servicios cloud de terceros y vulnerabilidades de seguridad en flujos con acceso privilegiado. El campo de la gobernanza de automatizaciones (Automation Governance) se proyecta como una de las áreas de mayor crecimiento en el período 2025–2028.

07 · CONCLUSIONES

7. Conclusiones

El RPA ha recorrido en menos de dos décadas un camino que va desde la imitación frágil de interfaces de usuario hasta la automatización inteligente capaz de razonar, adaptarse y colaborar con humanos en tiempo real. Este estado del arte permite extraer las siguientes conclusiones:

9. La distinción entre RPA, orquestación de datos e IA agencial se está diluyendo. Las organizaciones que traten estas tres disciplinas como silos separados perderán la oportunidad de construir sistemas de automatización verdaderamente robustos. La arquitectura ganadora es aquella que combina la accesibilidad de las herramientas low-code (n8n), la solidez del código (Prefect) y el razonamiento adaptativo de los agentes (LangGraph, CrewAI).
10. La observabilidad no es opcional. Un robot que falla silenciosamente es más peligroso que no tener robot. La implementación de logs estructurados, métricas de latencia y alertas debe ser parte del diseño inicial, no un añadido posterior.
11. La idempotencia y el manejo de errores son las habilidades de ingeniería más subvaloradas en RPA. La mayoría de los incidentes en producción no se deben a lógica incorrecta sino a la ausencia de gestión de excepciones, reintentos con backoff y validación de pre/postcondiciones.
12. El patrón human-in-the-loop es la estrategia de mitigación de riesgos más efectiva para flujos críticos. Automatizar la propuesta y reservar la confirmación para el humano permite capturar la eficiencia del RPA sin asumir el riesgo de errores autónomos en decisiones de alto impacto.
13. La gobernanza de automatizaciones es el siguiente frente de madurez. Las organizaciones que han escalado su portafolio de robots enfrentan ahora el desafío de mantener, auditar y controlar cientos de flujos interdependientes. Las metodologías y herramientas para este desafío están en sus primeras etapas de desarrollo.

Para el contexto del Prácticum DGTITD 2026, estas conclusiones sugieren que los retos de automatización se abordarán con mayor éxito cuando se partan de un proceso claramente definido, se priorice la observabilidad desde el día uno, se elija la herramienta apropiada según la complejidad del flujo (n8n para flujos simples, Prefect para lógica compleja) y se incorpore un agente de IA únicamente cuando las reglas no cubran todos los casos posibles.

08 · REFERENCIAS

8. Referencias

Aguirre, S., & Rodriguez, A. (2023). Automation of a business process using Robotic Process Automation (RPA) enhanced with Large Language Models. *Journal of Business Process Management*, 29(4), 812–834.

IBM. (2023). Banco Bradesco: AI-powered customer service automation. IBM Case Studies. Recuperado de <https://www.ibm.com/case-studies/bradesco>

Lacity, M. C., & Willcocks, L. P. (2016). A new approach to automating services. *MIT Sloan Management Review*, 58(1), 41–49.

Syed, R., Suriadi, S., Adams, M., Bandara, W., Leemans, S. J. J., Ouyang, C., ter Hofstede, A. H. M., van de Weerd, I., Wynn, M. T., & Reijers, H. A. (2020). Robotic Process Automation: Contemporary themes and challenges. *Computers in Industry*, 115, 103162.

van der Aalst, W. M. P., Bichler, M., & Heinzl, A. (2018). Robotic process automation. *Business & Information Systems Engineering*, 60(4), 269–272.